

PROJET RÉALISÉ AVEC AGENTIC AI

Systeme intelligent de gestion des demandes patients

Documentation technique et fonctionnelle · Workflow « Atelier_Projet »

RÉALISÉ PAR

Hiba Raqbi

Amal Ikhmim

Douaa Bouchama

Fatima Ezzahra Dennoune

Ayoub Zaddoug

Note de fidélité au workflow

Cette documentation est basée sur le JSON et les captures fournis. Écarts avec le brief initial : **(1)** l'entrée est un **Chat Input**, aucun node Tally n'existe dans le JSON ; **(2)** le workflow comporte **deux branches** (FAQ et rendez-vous) ; **(3)** il n'y a **pas de node de condition** « question trouvée ? » : cette distinction est portée par les prompts (voir §10).

1. Contexte du projet

Les établissements de santé reçoivent de nombreuses demandes de patients : questions pratiques, demandes de rendez-vous. Le projet met en place un système construit avec **Agentic AI** qui analyse chaque demande, l'oriente vers le bon traitement et déclenche les actions associées (email, notification, ticket). Les prompts prévoient des demandes en français, en darija ou en mélange des deux.

2. Problématique

Le traitement manuel impose au secrétariat de lire, classer et répondre à chaque message, y compris pour des questions récurrentes. Besoins identifiés :

- distinguer automatiquement une demande de rendez-vous d'une question FAQ ;
- répondre aux questions courantes à partir d'une source unique (MongoDB) sans inventer de contenu ;
- consigner les demandes sans réponse dans un ticket structuré ;
- informer le patient et le personnel du résultat d'une demande de rendez-vous.

3. Use case

Un patient écrit une demande, classée par le Smart Router. **Question FAQ** : l'agent cherche dans MongoDB, la réponse est envoyée par email (Gmail) et un ticket est préparé pour Notion. **Demande de rendez-vous** : l'agent extrait les informations, vérifie la disponibilité dans MongoDB, puis un email est envoyé au patient et un message Slack au personnel.

4. Solution proposée

Un workflow Agentic AI où des **agents spécialisés** (FAQ, rendez-vous, email, Slack) disposent chacun d'outils et d'un prompt précis. Ce n'est pas un simple chatbot : le système raisonne sur l'intention, interroge une base de données et agit sur des services externes (Gmail, Slack, Notion).

5. Architecture globale

Entrée Chat Input	Routage OpenRouter + Smart Router	Branche FAQ FAQ Agent, MongoDB, Email Agent, Gmail, Notion	Branche Rendez-vous Agent Rendez-vous, MongoDB, Gmail, Slack
-----------------------------	---	--	--

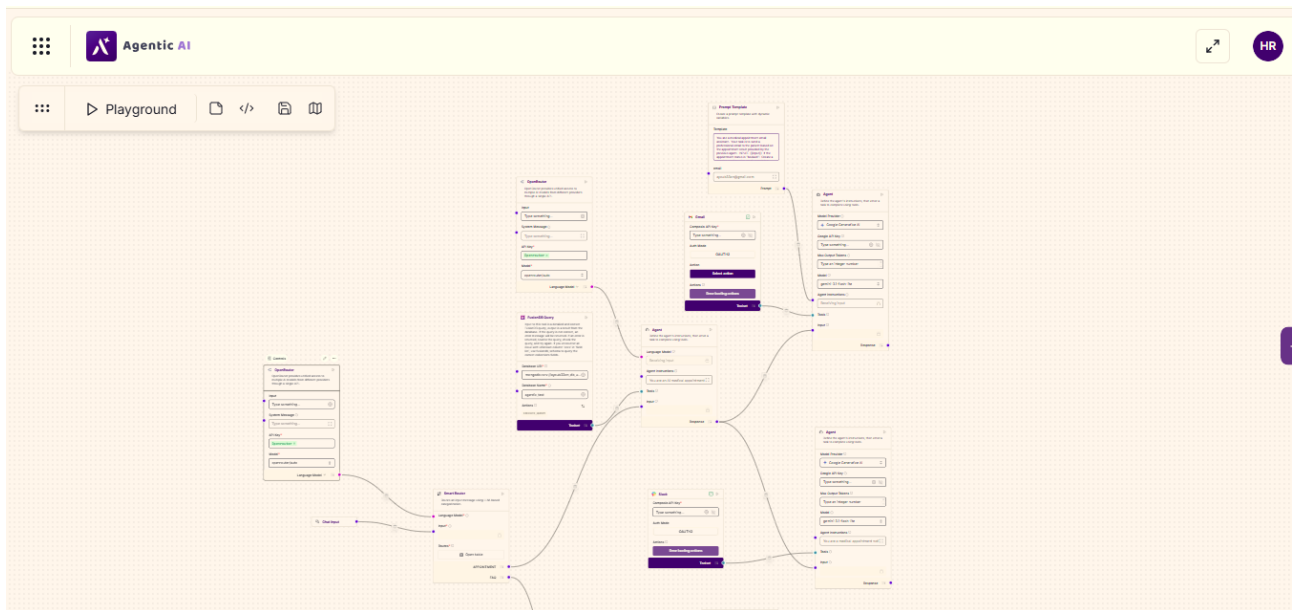


Figure 1 – Workflow Agentic AI : Chat Input, Smart Router (sorties APPOINTMENT et FAQ), FAQ Agent, agents Email et Slack.

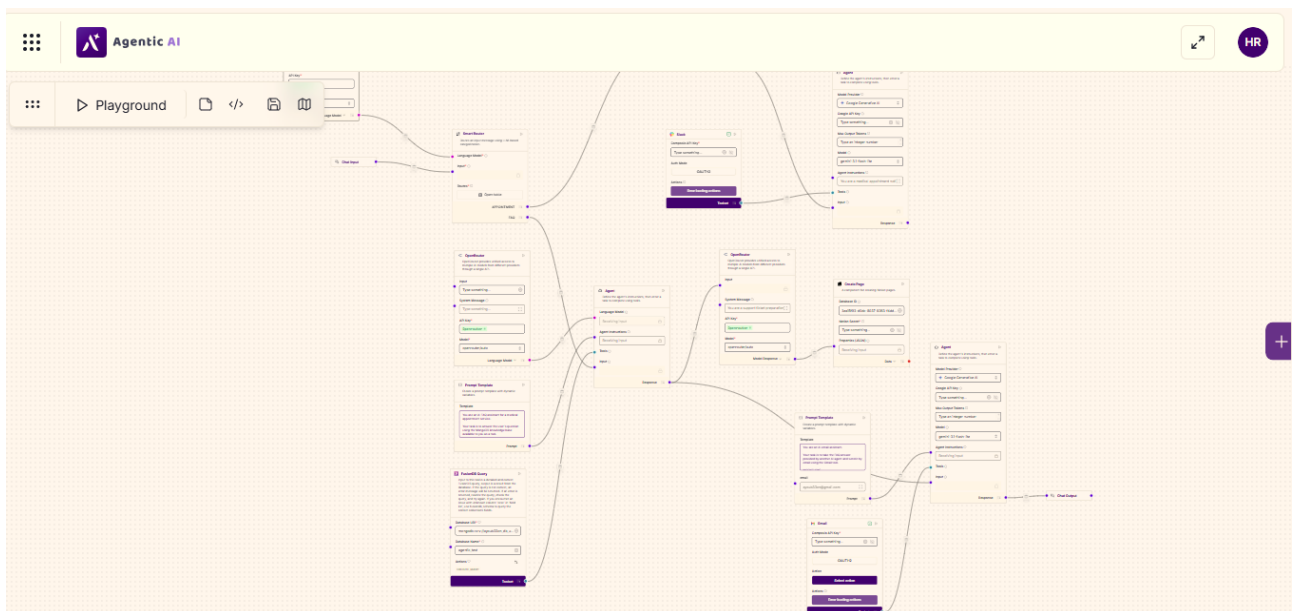


Figure 2 – Partie basse : branche ticket (OpenRouter vers Create Page Notion) et Chat Output.

6. Logique du workflow

- **Étape 1 - Entrée** : le message est saisi dans Chat Input.
- **Étape 2 - Routage** : le Smart Router classe la demande : APPOINTMENT ou FAQ.
- **Étape 3 - FAQ Agent** : interroge MongoDB (lecture seule), utilise le champ *answer* du document pertinent, sinon répond « Je n'ai pas trouvé cette information dans notre base de connaissances... ».
- **Étape 4 - Sorties FAQ** : la réponse est envoyée par l'Email Agent via Gmail puis affichée dans Chat Output ; en parallèle, un LLM prépare le JSON du ticket transmis à Notion.

- **Branche Rendez-vous** : l'Agent Rendez-vous interroge la collection *appointments* et renvoie un JSON de statut qui alimente un agent email (patient) et un agent Slack (personnel).

7. Description détaillée des nodes

Node	Rôle et traitement	Entrées	Sorties	Relations
Chat Input	Point d'entrée : reçoit le message du patient (Playground).	Texte saisi	Message	Smart Router
OpenRouter (routeur)	LLM de classification (<i>openrouter/auto</i> , température 0.7).	—	Modèle	Smart Router
Smart Router	Classe la demande en APPOINTMENT ou FAQ (sortie « else » désactivée).	Message + modèle	category_1 (APPOINTMENT), category_2 (FAQ)	APPOINTMENT : Agent Rendez-vous ; FAQ : FAQ Agent
OpenRouter (FAQ Agent)	LLM de l'agent FAQ (<i>openrouter/auto</i>).	—	Modèle	FAQ Agent
Prompt Template (FAQ)	Prompt système : recherche MongoDB en lecture seule, réponse en français, phrase de repli si rien trouvé.	Variable {{input}}	Prompt système	FAQ Agent
FusionDB Query (FAQ)	Outil MongoDB, opération <i>find</i> , base <i>agentic_test</i> , collection <i>Rendez_vous</i> .	Requête de l'agent	Documents FAQ	Outil du FAQ Agent
FAQ Agent	Cherche la réponse dans MongoDB et formule la réponse finale, ou la phrase de repli.	Message FAQ, prompt, modèle, outil	Réponse texte	Email Agent ; LLM de préparation du ticket
Prompt Template (Email FAQ)	Consignes : email professionnel fondé uniquement sur la réponse reçue, un seul email.	{{input}}, {email}	Prompt système	Email Agent
Gmail (FAQ)	Outil Gmail (Composio, OAuth2) pour l'envoi.	Appel de l'agent	Email envoyé	Outil de l'Email Agent
Email Agent	Rédige objet et corps puis envoie via Gmail (Google Generative AI, <i>gemini-3.1-flash-lite</i>).	Réponse du FAQ Agent	Confirmation	Chat Output
Chat Output	Affiche le résultat final dans le Playground.	Réponse de l'Email Agent	Message affiché	Fin de branche FAQ
OpenRouter (préparation ticket)	Prépare uniquement le JSON des propriétés du ticket (Name, Status « À traiter », Description). N'appelle aucun outil.	Réponse du FAQ Agent	JSON de propriétés	Create Page (Notion)
Create Page (Notion)	Crée une page dans la base Notion de tickets.	properties_json	Ticket Notion	Fin de branche ticket
OpenRouter (Rendez-vous)	LLM de l'agent de réservation (<i>openrouter/auto</i>).	—	Modèle	Agent Rendez-vous
Agent Rendez-vous	Extrait nom, service, motif, jour/date/heure/période ; vérifie la disponibilité (status « available ») ; renvoie un JSON <i>booked</i> / <i>unavailable</i> / <i>needs_information</i> .	Message APPOINTMENT	JSON de statut	Email Rendez-vous ; Slack
FusionDB Query (Rendez-vous)	Outil MongoDB, opération <i>find</i> , base <i>agentic_test</i> , collection <i>appointments</i> .	Requête de l'agent	Créneaux	Outil de l'Agent Rendez-vous

Node	Rôle et traitement	Entrées	Sorties	Relations
Agent Email Rendez-vous (+ Prompt Template + Gmail)	Envoie au patient un email de confirmation, d'indisponibilité ou de demande d'informations.	JSON de statut, {email}	Email envoyé	Sans sortie vers Chat Output
Agent Slack (+ Slack Composio)	Envoie un seul message Slack au personnel du cabinet (canal défini dans le prompt).	JSON de statut	Message Slack	Sans sortie vers Chat Output

8. Agentic AI

L'Agentic AI intervient là où une décision ou une action est nécessaire : classification de l'intention (Smart Router), choix des requêtes MongoDB (FAQ Agent, Agent Rendez-vous), rédaction et envoi d'emails, notification Slack, préparation d'un ticket. Elle reste encadrée : chaque agent a un prompt restrictif (« ne pas inventer », « un seul email ») et MongoDB est la source de vérité. Aucune autonomie en dehors de ces outils n'est configurée.

9. Flux de données

FAQ : message, Smart Router, FAQ Agent (échange avec MongoDB *Rendez_vous*), réponse, Email Agent, Gmail, Chat Output. En parallèle : réponse, LLM ticket, JSON, Notion.

Rendez-vous : message, Smart Router, Agent Rendez-vous (échange avec MongoDB *appointments*), JSON de statut, email patient (Gmail) et notification (Slack).

10. Gestion des cas

Cas	Traitement
Question trouvée	Le FAQ Agent s'appuie sur la réponse de MongoDB ; l'Email Agent l'envoie via Gmail ; le résultat apparaît dans Chat Output.
Question non trouvée	Le FAQ Agent renvoie la phrase de repli ; le LLM de préparation produit un ticket (Name, Status « À traiter », Description = demande d'origine) créé dans Notion.

Point d'attention : la réponse du FAQ Agent est connectée simultanément à l'Email Agent et à la préparation du ticket, sans node de condition. La séparation trouvé / non trouvé repose donc sur le texte de réponse et les prompts, pas sur un branchement explicite.

11. Outils et technologies

Outil	Usage dans le workflow
Agentic AI	Plateforme de conception du workflow (Playground)
Smart Router	Classification APPOINTMENT / FAQ
OpenRouter	LLM <i>openrouter/auto</i> : routeur, agents FAQ et rendez-vous, préparation du ticket
Google Generative AI	<i>gemini-3.1-flash-lite</i> : Email Agent, agent email Rendez-vous, agent Slack
MongoDB	Base <i>agentic_test</i> : collections <i>Rendez_vous</i> et <i>appointments</i>
Gmail (Composio)	Envoi d'emails (OAuth2)

Outil	Usage dans le workflow
Slack (Composio)	Notification du personnel (OAuth2)
Notion	Création de pages de ticket (Create Page)

Tally n'apparaît pas dans le JSON fourni.

12. Bénéfices fonctionnels

- Orientation automatique des demandes (FAQ / rendez-vous).
- Réponses aux questions courantes fondées sur la base MongoDB.
- Réponse par email et notification du personnel sans saisie manuelle.
- Tickets Notion structurés pour les demandes sans réponse.

13. Limites et évolutions possibles

Limites constatées dans la configuration actuelle

- Pas de node de condition explicite (voir §10).
- Le prompt FAQ cite la collection *medical_faq*, alors que l'outil est configuré sur *Rendez_vous* : à harmoniser.
- Le prompt de l'email FAQ contient *secretariat@example.com* en plus de la variable {email} : à vérifier pour un envoi au patient.
- Les outils MongoDB sont en opération *find* alors que l'Agent Rendez-vous doit « réserver » un créneau : la capacité d'écriture n'est pas démontrée par le JSON.
- Les branches Rendez-vous et ticket ne sont pas reliées à Chat Output.

Évolutions possibles (non implémentées)

- Ajouter un node de condition ou une sortie structurée « trouvé / non trouvé ».
- Brancher une entrée par formulaire (par exemple Tally) via webhook.
- Ajouter un suivi de statut des tickets.

14. Conclusion

Le workflow combine routage, agents spécialisés, base MongoDB et services externes pour traiter automatiquement les demandes patients, en gardant les réponses ancrées dans les données. Avec les ajustements du §13, il constitue une base solide d'automatisation du secrétariat par Agentic AI.